

Assignment 12: Parallel find_sum with Performance Evidence

Dr. Xiaoyuan Suo (xiaoyuansuo51@webster.edu)

Learning goals:

By the end, you will be able to:

- Implement a correct parallel reduction (sum) using Python multiprocessing (or threads if justified).
- Measure speedup, efficiency, and overhead using controlled experiments.
- Explain why scaling saturates (Amdahl + memory bandwidth + Python overhead).
- Produce reproducible performance evidence on a real machine.

Your task:

Generate a dataset of 5 rows × 10,000,000 columns (50 million numbers total) using a fixed seed. Implement:

1. **Sequential sum:** find_sum_serial(data)
2. **Parallel sum:** find_sum_parallel(data, workers)

Compute:

- Per-row sum
- Grand total sum (sum of row sums)

Your parallel result must match the serial result exactly (or within a stated tolerance if using floats—see “Correctness rules”).

Specific Requirements:

A. Required environment & run location

- Must run on our HPC cluster (Hellbender, use the account provided).
- You can check the correctness of your program locally, but the execution must be on Hellbender

B. Required data generation

- Use a fixed seed:
 - int32 in range [0, 100]
- You must store the array in a memory-efficient structure (e.g., NumPy array). Python lists of 50M elements are not allowed.

C. Required parallel method

Use multiprocessing with shared memory (not “send huge arrays to each process”).

- Acceptable: multiprocessing.shared_memory, numpy.frombuffer, ProcessPoolExecutor + shared memory.
- Not acceptable:
 - Parallelizing by slicing the array and pickling 50M numbers to workers.

- Using np.sum on the whole matrix as the only “solution” (you may use it for verification only).
- Using GPU/CuPy unless explicitly approved.

D. Required decomposition (forces thinking)

Your parallel version must use two-level reduction:

1. Each worker computes partial sums on assigned chunks.
2. A final reduction combines partial sums in the parent process.

You must implement your own chunking strategy (how you split work).

Deliverables

1) Code package

- sum_lab.py (main implementation)
- run_experiments.py (automated benchmarking script)

2) Short report (1 page max, PDF)

Must include the following sections:

(i) Correctness evidence

- Show serial and parallel totals for the same seed match.
- Include at least 2 additional tests:
 1. Small matrix (e.g., 5×1000) where you can print values and verify.
 2. Edge case test: all zeros OR all ones OR alternating pattern.

(ii) Performance plots (required)

Make these plots (no screenshots of terminal—use actual plots):

- Runtime vs workers (1, 2, 4, 8, ... up to max)
- Speedup vs workers
- Efficiency vs workers

3) Live check (5 minutes).

You will do part 1 as homework and we will complete part 2 in a lab session. During the lab session, you will:

- Run `python run_experiments.py --seed 123 --size full --workers 1 2 4`
- Explain your chunking and show where shared memory is used.
-

You must run:

- 3 trials per worker count and report median runtime.
- Worker counts: 1, 2, 4, 8, ... up to either:
 - number of physical cores, OR
 - where runtime no longer improves.

You must pin/record:

- OMP_NUM_THREADS=1, MKL_NUM_THREADS=1 (prevents hidden threading in NumPy)
- Note any CPU load/contention if on shared systems.

Grading rubric (100 pts)

Correctness (25)

- Serial correct (10)
- Parallel matches serial (10)
- Extra tests included (5)

Parallel design (25)

- Shared memory used correctly (10)
- Two-level reduction + chunking strategy (10)
- No forbidden approaches (5)

Performance methodology (25)

- 3 trials + median, proper worker sweep (10)
- Plots: runtime/speedup/efficiency (10)
- Reproducible scripts + logs (5)

Analysis & explanation (25)

- Overhead sources identified (10)
- Scaling limit explanation is accurate (10)
- Chunk size reasoning + tradeoffs (5)

Be ready for the 5-minute live check.